

End-to-End NLP Cheat Sheet

From Beginner to Advanced: A Comprehensive Guide to Natural Language Processing



Concepts



Techniques



Models



Naresh Edagotti
@Statfusionai

Introduction to NLP

What is NLP?

Natural Language Processing (NLP) is a field of AI that focuses on enabling computers to understand, interpret, and generate human language in a valuable way. It combines computational linguistics with machine learning and deep learning models.

Key Applications

 Chatbots & Virtual Assistants

 Machine Translation

 Sentiment Analysis

 Text Summarization

 Search Engines

 Speech Recognition

NLP Pipeline

The typical NLP pipeline involves several sequential steps to process text data:



Text Acquisition



Preprocessing



Feature Extraction



Modeling



Evaluation

NLP Python Libraries

Key Libraries Overview

spaCy

Industrial-strength NLP library optimized for production use with pre-trained models and support for 60+ languages.

```
pip install spacy
```

- ✓ Named Entity Recognition
- ✓ Dependency Parsing

NLTK

Classic library for education and research with extensive corpora and lexical resources.

```
pip install nltk
```

- ✓ Tokenization & Stemming
- ✓ Corpus Linguistics

gensim

Library for topic modeling and document similarity analysis with memory-efficient implementations.

```
pip install gensim
```

- ✓ Topic Modeling
- ✓ Word2Vec & Doc2Vec

Transformers

State-of-the-art library for pre-trained models like BERT, GPT, and T5 with PyTorch, TensorFlow, and JAX.

```
pip install transformers
```

- ✓ Transfer Learning
- ✓ Fine-tuning SOTA Models

When to Use Which Library

- ▶ **spaCy:** Production environments, fast processing, pre-trained models
- ▶ **NLTK:** Education, research, prototyping, linguistic analysis
- ▶ **gensim:** Topic modeling, document similarity, semantic analysis
- ▶ **Transformers:** State-of-the-art results, transfer learning, complex NLP tasks

Fundamental NLP Concepts

Core Text Processing Techniques

Tokenization

Breaking text into smaller units like words, sentences, or subwords.

```
"Hello world" → ["Hello", "world"]
```

Stop Words Removal

Removing common words that carry little semantic meaning.

```
["the", "a", "and", "in"] → []
```

Stemming & Lemmatization

Reducing words to their root form (stemming) or dictionary form (lemmatization).

```
"running" → "run"  
(stem/lemma)
```

POS Tagging

Assigning grammatical categories to words (noun, verb, adjective, etc.).

```
"run" → "VB" (verb)
```

Sequence Analysis Techniques

Sentence Detection

Identifying boundaries between sentences in a text.

```
"Hello. World." →  
["Hello.", "World."]
```

N-grams

Contiguous sequences of n items from a text (words, characters).

```
"natural language" →  
bigram
```

Chunking

Grouping adjacent tokens into phrases based on their POS tags.

```
"the red car" → noun  
phrase
```

Named Entity Recognition

Identifying and classifying named entities (people, places, organizations).

```
"John works at Google" →  
[PERSON, ORG]
```


Text Representation Techniques

☰ Basic Text Vectorization Methods



Bag of Words (BoW)

Represents text as a vector of word counts, ignoring grammar and word order but preserving multiplicity.

```
Vector = [count(w1),  
count(w2), ..., count(wn)]
```

"The cat sat" → [1, 1, 1, 0] for ["The", "cat", "sat", "dog"]

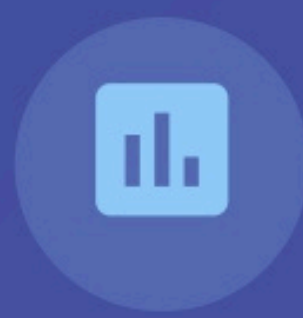


TF-IDF

Term Frequency-Inverse Document Frequency scores words by their importance in a document relative to a corpus.

```
TF-IDF = TF(t, d) × IDF(t)
```

"the" has high TF but low IDF, "algorithm" has higher TF-IDF



One-Hot Encoding

Represents each word as a binary vector with a single '1' at the position corresponding to that word.

```
"cat" = [0, 1, 0] for  
["dog", "cat", "sat"]
```

"cat" → [0, 1, 0], "dog" → [1, 0, 0]



Co-occurrence Matrix

Represents how frequently words appear together in a context, capturing semantic relationships.

```
M(i, j) = count(word_i,  
word_j in context)
```

"coffee" and "cup" have high co-occurrence in recipes

Word Embeddings (Intermediate Level)

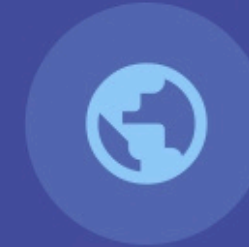
Key Word Embedding Models



Word2Vec

Neural network-based model that learns word associations from a large corpus of text. Words with similar meanings have similar vector representations.

Skip-gram: Predicts context words from target word
CBOW: Predicts target word from context words



GloVe

Global Vectors for Word Representation combines the benefits of Word2Vec and matrix factorization methods to capture both global and local semantics.

Learns word vectors by analyzing global co-occurrence statistics, capturing semantic relationships more effectively.



FastText

Extends Word2Vec by representing words as bags of character n-grams, enabling it to handle out-of-vocabulary words and morphologically rich languages.

Can generate vectors for words not seen during training by breaking them into character n-grams.



Contextual Embeddings

Advanced embeddings that generate different vector representations for the same word based on its context within a sentence, capturing polysemy.

ELMo, BERT, and GPT generate contextual embeddings that understand word meaning depends on context.

Static vs. Contextual Embeddings

- **Static:** Same vector for a word regardless of context (Word2Vec, GloVe, FastText)
- **Contextual:** Different vectors based on context (ELMo, BERT, GPT)
 - Static embeddings are simpler and faster to train
 - Contextual embeddings better capture meaning and improve performance on NLP tasks

Intermediate NLP Techniques

Core Text Analysis Methods



Named Entity Recognition

Identifies and classifies named entities in text into predefined categories like persons, organizations, locations, etc.

Uses sequence labeling models (CRF, BiLSTM-CRF) or transformer-based approaches

"Apple is based in Cupertino" → [ORG, LOC]

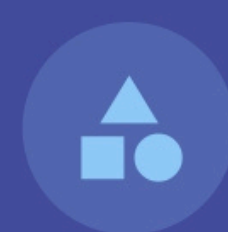


Dependency Parsing

Analyzes grammatical structure by establishing relationships between "head" words and words which modify them.

Creates dependency trees showing syntactic structure and relationships

"I saw a dog" → saw(dog), I(saw)



Text Classification

Assigns predefined categories to text documents based on their content using machine learning algorithms.

Uses Naive Bayes, SVM, or deep learning approaches for higher accuracy

"Great product!" → Positive sentiment



Sentiment Analysis

Determines the emotional tone behind a body of text, classifying it as positive, negative, or neutral.

Can be rule-based, using lexicons, or based on ML models like BERT

"The movie was terrible" → Negative



Advanced Text Processing Techniques



Similarity Measures

Quantifies how similar two pieces of text are using metrics like cosine similarity, Jaccard similarity, or Euclidean distance.

Word embeddings enable semantic similarity beyond just lexical overlap

"car" and "automobile" have high similarity despite different words



Chunking/Chinking

Groups adjacent tokens into meaningful phrases based on their POS tags, identifying noun phrases, verb phrases, etc.

Chunking: grouping tokens; Chinking: excluding tokens from groups

"The red car" → Noun phrase chunk

Advanced NLP Models and Architectures

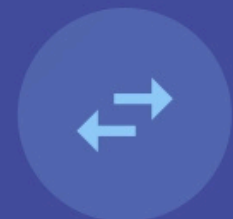
Neural Network Architectures



RNN / LSTM / GRU

Recurrent neural networks process sequential data by maintaining a hidden state that captures information from previous time steps.

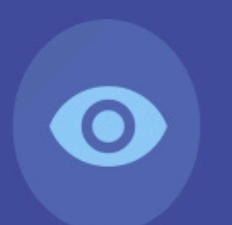
LSTM and GRU address vanishing gradient problem in basic RNNs through gating mechanisms



Seq2Seq Models

Sequence-to-sequence models use encoder-decoder architecture to transform input sequences to output sequences of different lengths.

Foundation for machine translation, text summarization, and conversation models



Attention Mechanism

Allows models to focus on specific parts of input sequence when generating each part of output sequence, improving performance on long sequences.

Self-attention enables each position to attend to all positions in the same sequence



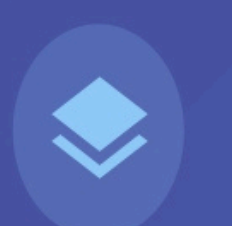
Transformer Models

Architecture based solely on attention mechanisms, eliminating recurrence and convolution for better parallelization and performance.

BERT, GPT, T5, and RoBERTa are popular transformer-based models



Contextual Embeddings & Large Language Models



Contextual Embeddings

Generate word representations that change based on context, capturing polysemy and improving understanding of word meaning in different situations.

ELMo uses bidirectional LSTMs; ULMFiT enables effective transfer learning for NLP



Large Language Models

Massive neural networks trained on enormous text corpora that can generate human-like text and perform diverse NLP tasks with prompting.

GPT-3/4, LLaMA, PaLM, and Claude demonstrate emergent abilities at scale

NLP Tasks and Applications

✦ Core NLP Tasks



Text Generation

Creating coherent and contextually relevant text based on input prompts or conditions.

Applications: Content creation, code generation, creative writing



Machine Translation

Automatically translating text from one language to another while preserving meaning.

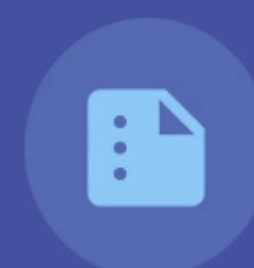
Applications: Google Translate, multilingual content localization



Question Answering

Automatically answering questions posed in natural language based on given context.

Applications: Virtual assistants, search engines, customer support

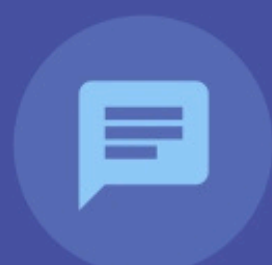


Text Summarization

Creating concise summaries of longer documents while preserving key information.

Applications: News digest, research paper abstracts, meeting notes

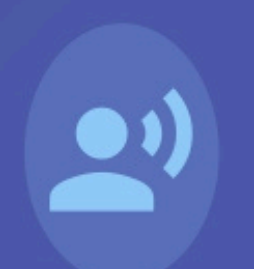
🗄 Advanced Applications



Conversational AI

Building systems that can engage in natural language conversations with humans.

Applications: Chatbots, virtual assistants, dialogue systems



Speech Recognition & TTS

Converting spoken language to text (ASR) and text to spoken language (TTS).

Applications: Voice assistants, accessibility tools, audio content creation

Practical Implementation

Frameworks and Tools

PyTorch & TensorFlow

Leading deep learning frameworks with extensive NLP capabilities, GPU acceleration, and pre-trained models.

PyTorch: Pythonic, dynamic graphs;
TensorFlow: production-ready, static graphs

Hugging Face

Comprehensive ecosystem with pre-trained models, datasets, and tokenizers for state-of-the-art NLP.

Transformers library: 100K+ models, Pipelines: easy NLP task implementation

Datasets and Benchmarks

Common Datasets

Standard datasets for training and evaluating NLP models across different tasks and domains.

IMDB (sentiment), SQuAD (QA), CoNLL (NER), WikiText (language modeling)

Evaluation Benchmarks

Comprehensive test suites for comparing model performance across multiple NLP tasks.

GLUE, SuperGLUE, HELM: multi-task benchmarks for model evaluation

Performance Metrics & Best Practices

Performance Metrics

Quantitative measures for evaluating model performance on different NLP tasks.

Accuracy, F1-score, BLEU, ROUGE, Perplexity: task-specific metrics

Best Practices

Proven techniques and methodologies for effective NLP model development and deployment.

Transfer learning, proper tokenization, hyperparameter tuning, model versioning